Ingeniería en Tecnologías de la Información y las Comunicaciones

Tecnologías Inalámbricas

12:00 p m

Control de Potencia Bluetooth

Profesor: Ricardo Alejandro Rodríguez Jiménez

Equipo:

Ramírez Rodríguez Martín de Jesús 23151193

Santacruz Rodarte Cristian Alexis 22150108

Pedroza Pérez Emmanuel 23151202

Fecha: 24 de febrero de 2026

Índice

Introducción 3

MIT App Inventor 4

Circuito 6

Código ESP32 7

Conclusiones 15

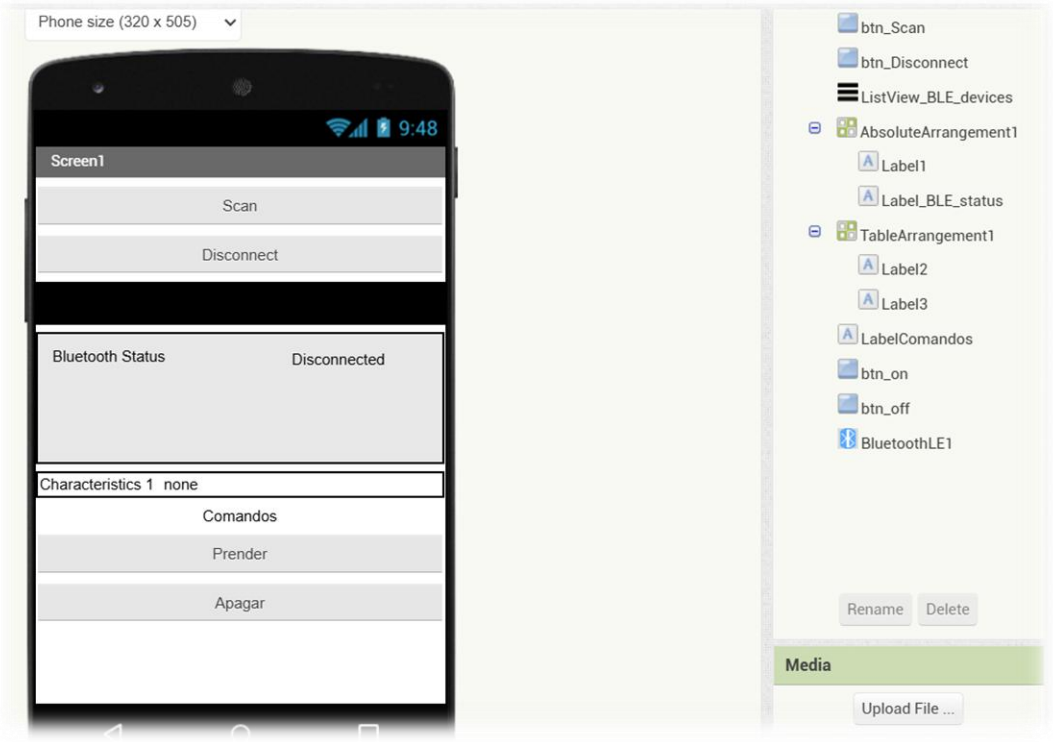
Enlace de Repositorio: 16

Introducción

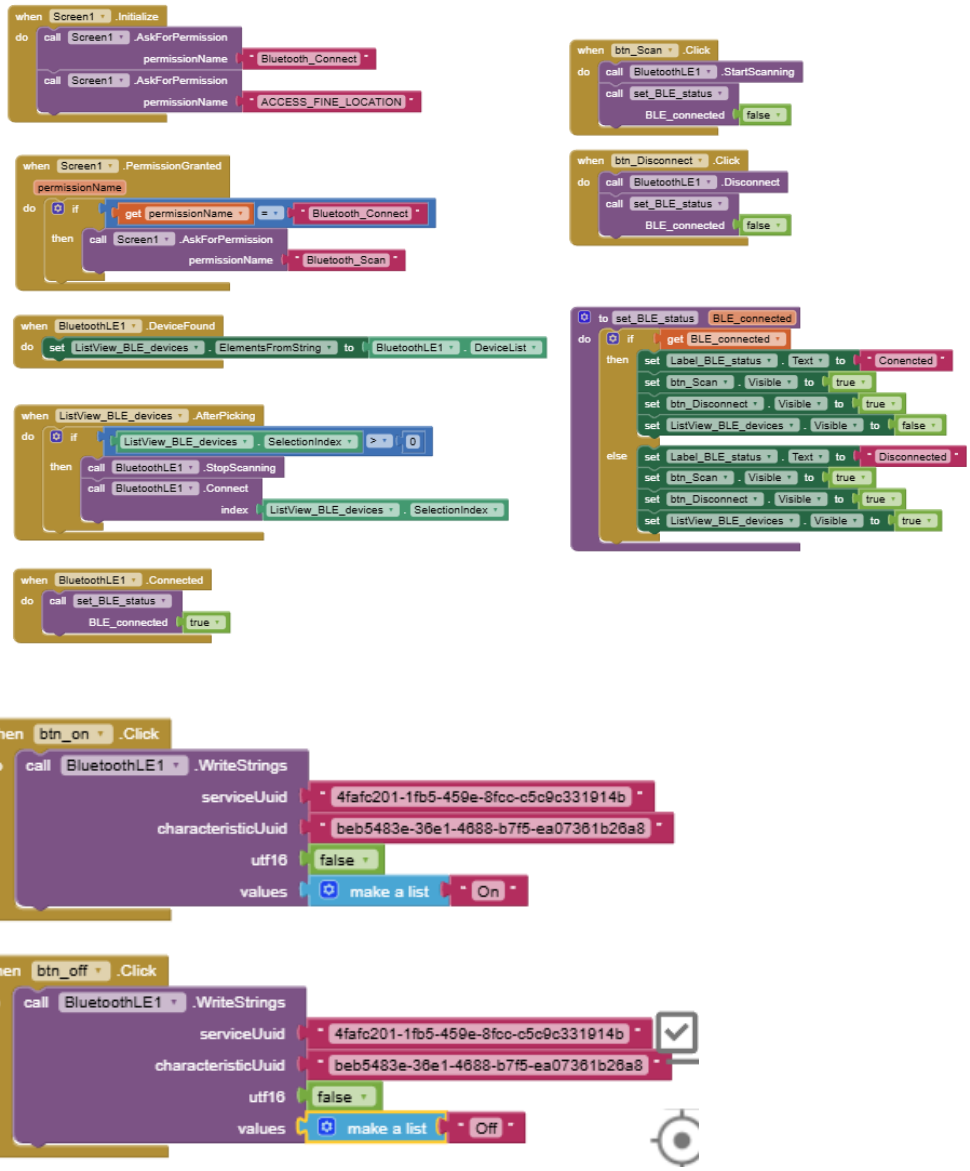
En este proyecto se diseñó una aplicación en MIT App Inventor que permite la interacción con un ESP32 mediante comunicación Bluetooth Low Energy (BLE) a un dispositivo móvil con el objetivo de controlar un foco conectado a un relevador.

La propuesta busca demostrar cómo, a través de una interfaz sencilla con botones de encendido y apagado, es posible gestionar de manera remota el funcionamiento de un dispositivo eléctrico convencional. El uso del relevador como intermediario garantiza la seguridad y la correcta adaptación de voltajes entre el microcontrolador y el foco, mientras que el ESP32 actúa como unidad central de procesamiento y comunicación.

MIT App Inventor

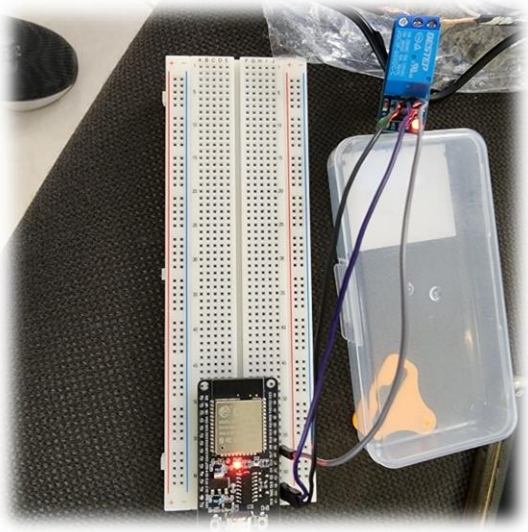


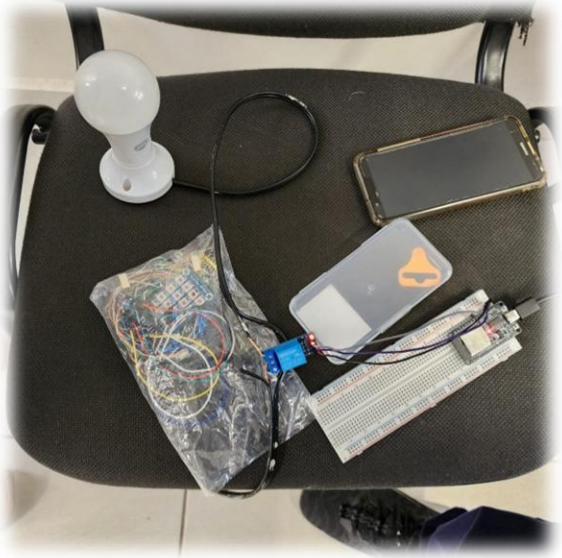
Esta nueva interfaz está hecha en base a la aplicación creada con anterioridad ESP32_BLE del mismo repositorio donde se encuentra este trabajo. Los cambios han sido mínimos, habiendo sólo eliminado un botón inferior y editando los otros dos botones de comando: btn_On y btn_Off, para encendido y apagado, respectivamente.



La estructura de bloques de la aplicación sigue la misma fórmula, habiendo retirado un botón y editando los otros dos, se han modificado para enviar una cadena de texto “On” y “Off” para ser recibidos por el ESP32 conectado al nuevo circuito y que respondan con las instrucciones del usuario en la aplicación móvil.

Circuito





Para construir el circuito que conectará con el circuito del ESP32 se cortó una parte de los cables del socket para conectarlo al relevador, y del relevador a la clavija, y finalmente, el relevador se conecta a la protoboard con el ESP32. Lo más difícil fue pelar el cable que era más grueso que los usados en el protoboard, y que no se usó cinta aislante, entonces los integrantes eran ligeramente electrocutados.

El ESP32 sigue conectado a un Led para mostrar su estado de conexión y luego al nuevo circuito del foco común.

Código ESP32

ESP32_BLE_controller.ino

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

#define DEVICE_NAME "No se conecten"
#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_UUID    "beb5483e-36e1-4688-b7f5-ea07361b26a8"
#define BRIGHTNESS   255   // Intensidad máxima (0-255)

// --- LED de estado (conexión BLE) ---
#define PIN_RED   21
#define PIN_GREEN 19
#define PIN_BLUE  18

// --- Relevador controlado por la app ---
int relayPin = 17;

BLEServer* pServer = NULL;
BLECharacteristic* pCharacteristic = NULL;
bool deviceConnected = false;
bool oldDeviceConnected = false;

// Función para escribir valores RGB en LED de estado
void rgbWrite(int r, int g, int b) {
  analogWrite(PIN_RED, r);
  analogWrite(PIN_GREEN, g);
  analogWrite(PIN_BLUE, b);
}

// Parpadeo rápido en LED de estado
void flashLED(int r, int g, int b, int count) {
  for (int i = 0; i < count; i++) {
    rgbWrite(r, g, b); delay(100);
    rgbWrite(0, 0, 0); delay(100);
```

ESP32_BLE_controller.ino

```
  }
}

// Función para controlar el relevador
void setRelay(String command) {
  if (command == "On") {
    digitalWrite(relayPin, HIGH);   // Activa el relevador
    Serial.println("Relevador ENCENDIDO");
  } else if (command == "Off") {
    digitalWrite(relayPin, LOW);    // Apaga el relevador
    Serial.println("Relevador APAGADO");
  }
}

// Callbacks BLE
class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) { deviceConnected = true; };
    void onDisconnect(BLEServer* pServer) { deviceConnected = false; }
};

// Callback para recibir datos desde la app
class MyCallbacks: public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
      String value = pCharacteristic->getValue();
      if (value.length() > 0) {
        Serial.print("Dato recibido: ");
        Serial.println(value);
        setRelay(value);   // Procesar comando "On" o "Off"
      }
    }
};

void setup() {
  Serial.begin(115200);
  Serial.println("Iniciando BLE...");
```

ESP32_BLE_controller.ino

```
  // Configurar pin del relevador
  pinMode(relayPin, OUTPUT);
  digitalWrite(relayPin, LOW); // Inicialmente apagado

  // BLE
  BLEDevice::init(DEVICE_NAME);
  pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  BLEService *pService = pServer->createService(SERVICE_UUID);

  pCharacteristic = pService->createCharacteristic(
                      CHAR_UUID,
                      BLECharacteristic::PROPERTY_READ |
                      BLECharacteristic::PROPERTY_WRITE |
                      BLECharacteristic::PROPERTY_NOTIFY
                    );
  pCharacteristic->addDescriptor(new BLE2902());
  pCharacteristic->setCallbacks(new MyCallbacks());

  pService->start();

  BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  pAdvertising->setScanResponse(true);
  pAdvertising->setMinPreferred(0x12);
  BLEDevice::startAdvertising();

  Serial.println("Esperando cliente...");
}

void loop() {
  // LED de estado (conexión BLE)
  if (deviceConnected && !oldDeviceConnected) {
```

```
    oldDeviceConnected = deviceConnected;
    Serial.println(">>> CONECTADO <<<");
    flashLED(0, BRIGHTNESS, 0, 5); // Verde parpadeo rápido
  }

  if (!deviceConnected && oldDeviceConnected) {
    Serial.println(">>> DESCONECTADO <<<");
    flashLED(BRIGHTNESS, 0, 0, 5); // Rojo parpadeo rápido
    delay(500);
    pServer->startAdvertising();
    oldDeviceConnected = deviceConnected;
  }

  if (deviceConnected) {
    rgbWrite(0, BRIGHTNESS, 0); // Verde fijo
  } else {
    rgbWrite(0, 0, BRIGHTNESS); // Azul fijo (esperando)
  }

  delay(10);
}
```

Se importan las librerías necesarias para las funciones Bluetooth LE

```
#include <BLEDevice.h>

#include <BLEServer.h>

#include <BLEUtils.h>

#include <BLE2902.h>
```

Definimos como se llamará el dispositivo y agregamos los identificadores de BLE además de el brillo máximo que puede tener cualquiera de los dos LED

```
#define DEVICE_NAME "No se conecten" // Nombre del dispositivo

#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"

#define CHAR_UUID    "beb5483e-36e1-4688-b7f5-ea07361b26a8"

#define BRIGHTNESS   255
```

Definimos que pines del ESP-32 serán los que controlarán los respectivos pines del LED RGB y se define el pin que controlara el relevador

```
// Led estado de conexión

#define PIN_RED   21

#define PIN_GREEN 19

#define PIN_BLUE  18

int relayPin = 17;
```

Se declaran dos punteros, pServer y pCharacteristic que apuntan a objetos necesarios para la comunicación BLE.

```
BLEServer* pServer = NULL;
BLECharacteristic* pCharacteristic = NULL;
```

Variables de tipo bool que guardan el estado de la conexión y si un dispositivo anterior esta conectado todavía o si fue desconectado

```
bool deviceConnected = false;
bool oldDeviceConnected = false;
```

Función para que el ESP-32 pueda controlar el LED RGB que indica el estado de la conexión BLE

```
// Led estado
void rgbWrite(int r, int g, int b) {
  analogWrite(PIN_RED, r);
  analogWrite(PIN_GREEN, g);
  analogWrite(PIN_BLUE, b);
}
```

Función para activar o desactivar el relevador basado en el comando que recibe

```
void setRelay(String command) {
  if (command == "On") {
    digitalWrite(relayPin, HIGH);   // Activa el relevador
    Serial.println("Relevador ENCENDIDO");
  } else if (command == "Off") {
    digitalWrite(relayPin, LOW);    // Apaga el relevador
    Serial.println("Relevador APAGADO");
```

```
  }
}
```

Esta clase permite saber si actualmente hay algún dispositivo conectado.

```
// Callbacks BLE
class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) { deviceConnected = true; };
    void onDisconnect(BLEServer* pServer) { deviceConnected = false; }
};
```

Esta clase utiliza onWrite para que solo sea ejecutada si recibe el valor char de una letra. Despues de obtener el valor, llama al método setColor para que encienda el LED RGB del color solicitado

```
class MyCallbacks: public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
      String value = pCharacteristic->getValue();
      if (value.length() > 0) {
        Serial.print("Dato recibido: ");
        Serial.println(value);
        setRelay(value);
    }
};

void setup() {
```

Configuracion de la comunicacion serial y de los pines del LED

```
  Serial.begin(115200);
  Serial.println("Iniciando BLE...");
```

```
  pinMode(relayPin, OUTPUT);
 digitalWrite(relayPin, LOW);
```

Indica que se inicie el dispositivo BLE con el nombre asignado.

```
 BLEDevice::init(DEVICE_NAME);
```

Indica que se haga un servidor BLE en el ESP32, además se le asigna la función de Callback para comprobar conexiones y desconexiones

```
 pServer = BLEDevice::createServer();
 pServer->setCallbacks(new MyServerCallbacks());
```

Indica que se haga el servicio BLE con el UUID asignado

```
 BLEService *pService = pServer->createService(SERVICE_UUID);
```

Realiza una característica o variable para que se pueda leer, escribir y además que notifique cuando el valor de la característica cambie

```
 pCharacteristic = pService->createCharacteristic(
                    CHAR_UUID,
                    BLECharacteristic::PROPERTY_READ |
                    BLECharacteristic::PROPERTY_WRITE |
                    BLECharacteristic::PROPERTY_NOTIFY
                  );
```

Añade el descriptor BLE2902 para permitir las notificaciones y se asigna el método MyCallbacks() para controlar el LED

```
 pCharacteristic->addDescriptor(new BLE2902());
 pCharacteristic->setCallbacks(new MyCallbacks());
```

Activa el servicio BLE

```
  pService->start();
```

Inicia el advertising del dispositivo BLE añadiendo el UUID, para que pueda responder al escaneo y se agrega un parámetro que mejora la conexión.

```
  BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  pAdvertising->setScanResponse(true);
  pAdvertising->setMinPreferred(0x12);
```

El dispositivo comienza a anunciarse para poder ser encontrado

```
  BLEDevice::startAdvertising();

  Serial.println("Esperando cliente...");
}
void loop() {
```

Revisa si hay un dispositivo que se acabe de conectar y no había una conexión anterior, cambia el estado del dispositivo anterior al conectado actualmente y manda instrucción al LED de estado para que indique que se conectó un dispositivo parpadeando 5 veces

```
  // led estado
  if (deviceConnected && !oldDeviceConnected) {
    oldDeviceConnected = deviceConnected;
    Serial.println(">>> CONECTADO <<<");
    flashLED(0, BRIGHTNESS, 0, 5);
```

```
  }
```

Revisa si se desconectó el dispositivo, da la instrucción al LED para indicar la desconexión con un color rojo y reinicia la función de anuncio para que se pueda encontrar de nuevo

```
  if (!deviceConnected && oldDeviceConnected) {
    Serial.println(">>> DESCONECTADO <<<");
    flashLED(BRIGHTNESS, 0, 0, 5);
    delay(500);
    pServer->startAdvertising();
    oldDeviceConnected = deviceConnected;
  }
```

Si no hay cambios en la conexión del dispositivo, si hay conexión mantiene un color verde y si no hay conexión mantiene un color azul

```
  if (deviceConnected) {
    rgbWrite(0, BRIGHTNESS, 0);
  } else {
    rgbWrite(0, 0, BRIGHTNESS);
  }
  delay(10);
}
```

El código preparado para el microcontrolador no ha tenido más percances, sólo se han cambiado las cadenas de texto a recibir y su debida respuesta al encendido o al apagado.

Conclusiones

Ramírez Rodríguez Martín de Jesús:

En este proyecto se logró nuevamente integrar de manera efectiva una aplicación móvil desarrollada en MIT App Inventor con un ESP32, pero ahora también con un relevador para controlar un foco. La comunicación mediante Bluetooth LE permitió establecer un enlace confiable entre el dispositivo móvil y el microcontrolador, garantizando la ejecución de las órdenes de encendido y apagado.

Este ha sido un paso más que sienta las bases para futuras mejoras, como la incorporación de interruptores físicos o la ampliación hacia sistemas más complejos de automatización en el Internet de las Cosas.

Santacruz Rodarte Cristian Alexis:

En conclusión, la práctica de potencias con blueetooth nos ayudó a conocer los cimientos básicos del control al hacer uso de Blueetooth en dispositivos que pueden ser usados en la vida cotidiana, en este caso fue implementada el uso de una aplicación móvil para la manipulación de un foco LED con control inalámbrico.

Pedroza Pérez Emmanuel:

Para concluir, a comparación de la práctica anterior es interesante por el uso del relevador de manera remota, permitiéndonos hacer una cantidad enorme de aplicaciones útiles que solucionen problemáticas de la vida diaria, demostrando la utilidad del bluetooth LE.

Enlace de Repositorio:

Aquí se encontrará un enlace para redirigirle al repositorio donde se presentan el código de ESP32, el archivo creado de MIT App Inventor, el video de evidencias y la misma documentación:

https://github.com/MartinITAAgs/Tecnolog-as-Inal-mbricas-12pm/tree/main/ESP32_focazo